

MEDICAL AI PROJECT · UNIVERSITY FINAL PROJECT

PNEUMONIA DETECTION FROM CHEST X-RAYS

Using Convolutional Neural Networks & Transfer Learning

Technical Report & Learning Journal

September 2025 – January 2026

86.1%

FINAL TEST ACCURACY

98%+

PNEUMONIA RECALL

50%

FEWER FALSE ALARMS

Dataset: 5,856 Chest X-rays (Kaggle) • **Framework:** PyTorch (CPU) • **Models:** Baseline CNN & ResNet18

0

1	Project Overview & Objective	3
1.1	The Problem – Why This Matters	3
1.2	Dataset & Technical Stack	3
2	Data Preparation & Key Challenge	3
2.1	The Validation Set Problem	3
2.2	Image Preprocessing Pipeline	4
3	Model 1 – Baseline CNN (From Scratch)	4
3.1	Architecture Design	4
3.2	Class Imbalance Strategy	4
3.3	Training Configuration	5
3.4	Baseline Results	5
4	The Solution – Transfer Learning with ResNet18	5
4.1	Why Transfer Learning?	5
4.2	Implementation: 4-Step Workflow	6
4.3	ResNet18 Training Details	6
5	Results & Performance Comparison	6
5.1	Quantitative Improvement	7
5.2	Confusion Matrix Comparison	7
5.3	Clinical Simulation: 1,000 Patients	7
6	Model Explainability – Grad-CAM	7
6.1	Why Explainability is Non-Negotiable	8
6.2	Grad-CAM Implementation	8
6.3	Clinical Validation from Grad-CAM	8
7	Technical Concepts – Key Learning Notes	8
7.1	The Training Loop in Detail	8
7.2	Class Weights – The Medical Priority Setting	9
7.3	The Forward Pass: What Happens Inside ResNet18	9
7.4	Web Application: Full-Stack Deployment (Bonus)	9
8	Technical Challenges & Solutions	10
9	Clinical Relevance & Future Roadmap	10
9.1	Real-World Applications	10

9.2	Ethics & Limitations	10
9.3	Future Work Roadmap	11
10	Key Learnings & Project Achievements	11
10.1	Technical Insights	11
10.2	Medical AI Insights	11
10.3	Project Achievements Summary	12

1

Objective

Build an AI system that accurately classifies chest X-rays as **NORMAL** or **PNEUMONIA** to assist medical diagnosis, reducing the workload of radiologists and improving early detection rates.

1.1 The Problem – Why This Matters

Pneumonia kills approximately **2.5 million people annually** (WHO, 2023), making it the leading infectious cause of death in children worldwide. The current diagnostic workflow has critical bottlenecks:

- Manual X-ray interpretation is **time-consuming and subjective**, varying between radiologists
- **Shortage of specialists** in developing regions means delayed or no diagnosis
- **Human fatigue** during high-volume shifts increases diagnostic errors
- **Delayed diagnosis** directly worsens patient outcomes and increases mortality risk

1.2 Dataset & Technical Stack

Dataset (Kaggle – Chest X-Ray Images):

- 5,856 chest X-ray images total
- Binary classification: Normal vs. Pneumonia
- **Severe class imbalance:** 73% Pneumonia / 27% Normal
- Original Kaggle split had a critically inadequate validation set (8 images per class)

Technical Stack:

- **Language:** Python 3.11
- **Framework:** PyTorch 2.0+
- **Environment:** Jupyter Notebooks
- **Hardware:** CPU-only training (no GPU required)
- **Models:** Baseline CNN + ResNet18 (Transfer Learning)

2

2.1 The Validation Set Problem

Critical Issue Identified

The original Kaggle split provided only **8 images per class** for validation (16 total). This is statistically insufficient to reliably monitor overfitting or tune hyperparameters. Training on this data would produce misleading performance estimates.

Data distribution comparison:

Subset	Normal	Pneumonia	Total	Status
— Original Kaggle Split —				
Training	1,341	3,875	5,216	
Validation	8	8	16	PROBLEM: Too small!
Testing	234	390	624	
— Our Fixed Stratified Split —				
Training	1,073	3,101	4,174	
Validation	276	782	1,058	FIXED: 20% stratified
Testing	234	390	624	(unchanged)

Our Solution: Stratified Split

We merged the training and validation sets, then applied a **20% stratified split** that preserves class ratios. This increased validation size from 16 to **1,058 images**, providing statistically meaningful training evaluation.

2.2 Image Preprocessing Pipeline

All images passed through a consistent preprocessing pipeline:

transforms.Compose – Training Pipeline

```
train_transform = transforms.Compose([ transforms.Resize((224, 224)), # Standardize
size transforms.RandomHorizontalFlip(p=0.5), # Augmentation
transforms.ToTensor(), #
Convert to tensor transforms.Normalize(mean=[0.5], std=[0.5]) # Scale to [-1, 1] ])
```

Key preprocessing decisions:

- **224×224 resize**: Required by ResNet18 architecture
- **Random horizontal flip**: Data augmentation to prevent overfitting (training only)
- **Normalization**: Ensures consistent pixel value distribution across all inputs

3

3.1 Architecture Design

The baseline CNN is a 3-block convolutional network trained entirely from random initialization:

Input (224×224×3) → **Block 1** Conv(3→16) + ReLU + MaxPool → **Block 2**
Conv(16→32) + ReLU + MaxPool
→ **Block 3** Conv(32→64) + ReLU + MaxPool → **Classifier** Flatten + Dense(128) +
Dropout(0.5) → **Output** Sigmoid (2 classes)

3.2 Class Imbalance Strategy

With 73% Pneumonia and 27% Normal images, a naive model simply predicts “Pneumonia” for everything. Three strategies were applied:

- **Weighted Loss Function:** Normal classification errors are penalized 3× more:

$$w_{\text{Normal}} = \frac{N}{2 \cdot N_{\text{Normal}}} = 1.945, \quad w_{\text{Pneumonia}} = \frac{N}{2 \cdot N_{\text{Pneumonia}}} = 0.673$$

- **Data Augmentation:** Random horizontal flips during training
- **Dropout (50%):** Regularization to reduce overfitting

3.3 Training Configuration

Parameter	Value
Epochs	10
Optimizer	Adam (LR = 0.001)
Loss	CrossEntropyLoss with class weights
Total parameters	~6.5M (all trainable)
Training time	~15 minutes on CPU

3.4 Baseline Results

Metric	Value	Assessment	Clinical Implication
Validation Accuracy	97.6%	Misleading	Overfitting artifact
Test Accuracy	79.0%	Low	18.6% generalization gap
Pneumonia Recall	98.5%	Excellent	Almost no missed cases
Normal Recall	47.0%	Poor	53% of healthy mislabeled
False Positives	154	High	154 healthy patients flagged sick

		Predicted	
		Normal	Pneumonia
Actual	Normal	80	154
	Pneumonia	6	384

Baseline CNN Confusion Matrix (n=624)

Clinical Interpretation

The Baseline CNN is “**trigger happy**”: it catches almost all pneumonia (good) but creates **154 false alarms** in healthy patients. This generates unnecessary anxiety, testing costs, and wastes clinical resources.

4

4.1 Why Transfer Learning?

The Core Insight

Instead of learning from 5,856 images from scratch, we leverage ResNet18 pre-trained on **1.2 million ImageNet images** — a 200× larger knowledge base. The model already knows how to detect edges, textures, shapes, and patterns.

The mechanic analogy:

- **Baseline CNN** = New apprentice learning car mechanics from zero manuals

- **Transfer Learning** = Bicycle expert applying knowledge of wheels and gears to understand car parts — faster, fewer mistakes

How ResNet18's knowledge transfers to X-rays:

What ResNet18 Knows	X-ray Feature	Benefit
Edge detection	Rib cage boundaries	Identifies lung borders
Texture recognition	Lung parenchyma	Distinguishes normal vs. inflamed tissue
Object detection	White consolidation blobs	Identifies pneumonia patches
Background modeling	Clear lung areas	Recognizes normal lung appearance

4.2 Implementation: 4-Step Workflow

1. **Load Pre-trained ResNet18:** Download model with ImageNet weights (11.2M parameters encoding rich visual knowledge)
2. **Freeze All Convolutional Layers:** Lock backbone to preserve ImageNet knowledge

Freezing Layers

```
for param in resnet_model.parameters(): param.requires_grad = False # Preserve
all ImageNet knowledge
```

3. **Replace Final Classifier Head:** Remove the 1,000-class output, replace with 2-class head:

New Classification Head

```
num_features = resnet_model.fc.in_features # = 512
resnet_model.fc = nn.Linear(num_features, 2) # 512 -> Normal/Pneumonia
```

4. **Train Only Final Layer:** Update only 1,026 parameters instead of 11M — fast, stable, resistant to overfitting

4.3 ResNet18 Training Details

Parameter	Value
Base model	ResNet18 (ImageNet pre-trained)
Total parameters	11.2M
Trainable parameters	1,026 (final layer only)
Frozen parameters	11,176,000
Optimizer	Adam (LR = 1e-3, only final layer)
Epochs	10
Training time	~25 minutes on CPU
Model saving	Best validation loss checkpoint

5.1 Quantitative Improvement

Metric	Baseline CNN	ResNet18	Improvement	Significance
Test Accuracy	79.0%	86.1%	+7.1%	Reliable generalization
Normal Recall	47%	67%	+20%	Critical clinical gain
Pneumonia Recall	98.5%	97.4%	-1.1%	Safety maintained
False Positives	154	77	-50%	Half the false alarms
False Negatives	6	10	+4	Slight safety tradeoff
Training time	15 min	25 min	+10 min	Acceptable overhead

Summary metrics row:

86.1% Test Accuracy	67% Normal Recall	77 False Positives	97.4% Pneumonia Recall
---------------------------------------	-------------------------------------	--------------------------------------	--

5.2 Confusion Matrix Comparison

Baseline CNN (n=624)				ResNet18 (n=624)			
		Predicted				Predicted	
		Normal	Pneumonia			Normal	Pneumonia
Actual	Normal	80	154 ↓	Actual	Normal	157 ↑	77
	Pneumonia	6	384		Pneumonia	10	380
Normal Recall = 34%				Normal Recall = 67% (+20%)			

Key Conclusion

ResNet18 **doubles** the number of healthy patients correctly cleared (80 → 157) while **halving** unnecessary false alarms (154 → 77). Pneumonia detection remains near-perfect (380/390 cases caught). This is the right balance for a clinical triage tool.

5.3 Clinical Simulation: 1,000 Patients

Scenario	Baseline CNN	ResNet18
Healthy patients correctly sent home	~340	~670 (+97%)
False alarms (healthy flagged sick)	~660	~330 (-50%)
Pneumonia cases detected	~984	~974
Pneumonia missed (safety risk)	~16	~26

6.1 Why Explainability is Non-Negotiable

The Black Box Problem in Medicine

Standard “black box” AI is not trusted in high-stakes clinical settings. A radiologist cannot verify or validate a diagnosis without understanding *why* the model reached a conclusion. Clinical adoption requires explainability as a mandatory feature, not an optional add-on.

6.2 Grad-CAM Implementation

Gradient-weighted Class Activation Mapping (Grad-CAM) generates a heatmap overlay showing which image regions most influenced the model’s prediction:

1. Compute gradients of the predicted class score with respect to the last convolutional feature maps
2. Global average pool the gradients to get importance weights per feature map
3. Produce weighted combination of feature maps, apply ReLU
4. Resize and overlay on original X-ray as a color heatmap

6.3 Clinical Validation from Grad-CAM

Grad-CAM confirms the model behaves correctly by medical standards:

- **Pneumonia cases:** High activation concentrated in **lung parenchyma** – specifically in consolidation areas (white blobs characteristic of pneumonia)
- **Normal cases:** **Symmetric, diffuse** activation across both lungs – no localized hot spots
- **Critical check:** The model does **not** focus on equipment labels, wires, or bone structures – only lung tissue

Clinical Trust Result

Grad-CAM reveals the model examines the right anatomical regions. This allows radiologists to **verify the AI’s reasoning** in seconds, building trust and enabling safe clinical integration. It also helps clinicians spot subtle consolidation patterns they might otherwise miss.

7

7.1 The Training Loop in Detail

Every training step follows the same 5-operation cycle, which is critical to understand:

The Core Training Loop

```
for images, labels in loader: optimizer.zero_grad() # [1] Clear gradients from
previous batch outputs = model(images) # [2] Forward pass: compute predictions loss
= criterion(outputs, labels) # [3] Calculate error loss.backward() # [4] Backprop:
compute gradients optimizer.step() # [5] Update weights
```

Why `optimizer.zero_grad()` is essential:

Gradients *accumulate* by default in PyTorch. Without resetting each batch:

- Batch 1 error (+5) + Batch 2 error (−3) = accumulated gradient (+2)
- The update uses the wrong, mixed gradient signal
- This corrupts learning and leads to divergence

“Aha!” Moment – The Scoreboard Analogy

Think of gradients as a basketball scoreboard. `zero_grad()` wipes the scoreboard clean before each play. Without wiping it, feedback from 3 plays ago corrupts the current player's update. Each batch needs a fresh, isolated learning signal.

7.2 Class Weights – The Medical Priority Setting

The weighted loss function encodes medical knowledge directly into training:

$$\mathcal{L} = - \sum_c w_c \cdot y_c \cdot \log(\hat{y}_c)$$

Where $w_{\text{Normal}} = 1.945$ and $w_{\text{Pneumonia}} = 0.673$ means:

- Misclassifying a **Normal** patient as sick costs **2.9× more** than the reverse
- This biases the model to reduce false positives (unnecessary treatments)
- However, class weights alone cannot overcome a fundamentally biased model – dataset quality matters equally

“Aha!” Moment

“Class weights alone can't fix fundamental model bias!” – Transfer learning combined with class weights gives the best balance.

7.3 The Forward Pass: What Happens Inside ResNet18

When an X-ray is processed:

1. **Layer 1–4 (Frozen):** Extract hierarchical features – edges → textures → shapes → objects
2. **Global Average Pooling:** Compress spatial features to a 512-dimensional vector
3. **Final FC Layer (Trainable):** Map 512 features to 2 class scores
4. **Softmax:** Convert scores to probabilities summing to 1.0

7.4 Web Application: Full-Stack Deployment (Bonus)

The project was extended to a web interface using React frontend and FastAPI backend:

FastAPI Prediction Endpoint

```
@app.post("/api/analyze") async def analyze(file: UploadFile = File(...)):
    contents = await file.read() image = Image.open(io.BytesIO(contents)).convert('RGB')
    prediction, confidence, probs = predict_image(model, image, file.filename) return
```

```
{"diagnosis": prediction, "confidence": round(confidence, 1), "severity": "high"
if confidence > 85 else "moderate"}
```

Key architecture decisions:

- **CORS middleware:** Required to allow cross-origin browser requests between frontend and backend
- **Same preprocessing:** The exact transforms used in training must be applied at inference
- **Port consistency:** Frontend must target the correct backend port (8000, not 5000)

8

Challenge	Problem	Solution
Class Imbalance	73% Pneumonia / 27% Normal skews naive models	Weighted CrossEntropyLoss: Normal errors penalized 2.9×
Inadequate Val Set	Original Kaggle: only 8 images per class	Merged + 80/20 stratified split → 1,058 val images
Overfitting	Baseline showed 97.6% val vs. 79% test (18.6% gap)	Frozen ResNet18 backbone prevents feature destruction
Clinical Safety	Recall must be >95% – missing pneumonia is life-threatening	Asymmetric class weights prioritize recall over precision
Port Mismatch	Frontend targeting wrong backend port	Align fetch URL with unicorn port setting

9

9.1 Real-World Applications

- **Triage System:** Automatically prioritize high-risk X-rays in emergency rooms for immediate review
- **Second Opinion:** Automated double-check for fatigued radiologists during overnight shifts
- **Remote Diagnosis:** Deploy in rural areas with X-ray equipment but no onsite specialists
- **Medical Education:** Training tool for students to learn X-ray pattern recognition

9.2 Ethics & Limitations

- **Assistive, not autonomous:** Final diagnosis must always be human-verified – AI as a tool, not a replacement
- **Bias validation:** Model must be tested across diverse populations (age, sex, ethnicity)
- **Data privacy:** All patient data must be anonymized and HIPAA/GDPR compliant
- **Mandatory explainability:** Grad-CAM or similar techniques are required for clinical trust

9.3 Future Work Roadmap

Phase 1 (0–6 months):

1. Unfreeze deeper ResNet layers and fine-tune with lower learning rate (1e-4)
2. External validation on hospital datasets from different geographic regions
3. Subtype classification: Bacterial vs. Viral vs. COVID-19 pneumonia
4. Deploy web interface via Streamlit or Flask for clinical demonstration

Phase 2 (6–18 months):

1. Multi-modal AI: Combine X-ray data with patient history and EMR records
2. Severity scoring: Predict infection severity, not just binary presence/absence
3. Treatment assistance: Recommend antibiotic vs. supportive care based on classification
4. Cloud API deployment for low-resource global regions

10

10.1 Technical Insights

- **Transfer learning dramatically outperforms training from scratch** (+7.1% accuracy, +20% normal recall) with only 1,026 trainable parameters versus 6.5M
- **Class imbalance must be addressed explicitly** – weighted loss is essential; ignoring it causes the model to predict the majority class blindly
- **Explainability is mandatory in medical AI** – Grad-CAM is not optional; without it, clinical adoption is impossible regardless of accuracy
- **Data quality matters as much as model architecture** – fixing the validation split was as impactful as changing the model

10.2 Medical AI Insights

- **Recall is the primary safety metric** in pneumonia detection – a missed case (false negative) is far more dangerous than a false alarm (false positive)
- **Reducing false alarms (154 → 77) has tangible human impact** – fewer healthy patients experience unnecessary anxiety, tests, and costs
- **Incremental accuracy gains scale to massive impact** – a 7% improvement across millions of screenings saves thousands of lives annually

10.3 Project Achievements Summary

Achievement	Detail	Status
Working detection system	Complete pipeline from X-rays to classification	✓ Complete
Baseline CNN	79% test accuracy, 3-block architecture	✓ Complete
Transfer learning model	86.1% accuracy, ResNet18	✓ Complete
Clinical safety	Pneumonia recall >97%	✓ Achieved
False alarm reduction	50% fewer false positives	✓ Achieved
Grad-CAM explainability	Heatmap visualization implemented	✓ Complete
Web application	React + FastAPI interface	✓ Complete

Core Philosophy

“AI won’t replace radiologists, but radiologists using AI will replace those not using it.”

— Medical AI Principle

This project demonstrates that accessible, CPU-trainable, explainable AI can meaningfully improve clinical triage at scale.